

ABBA2 Midpoint

Explicit duplicated-space step with arithmetic projection

1 Definition

For $\dot{z} = f(t, z)$, $z \in \mathbb{R}^m$, duplicate the accepted state on the diagonal, $u_0 = v_0 = z_n$, and write $s = h/2$. The endpoint-time ABBA stages are

$$u_1 = u_0 + sf(t_n, v_0), \quad v_1 = v_0 + sf(t_n, u_1), \quad (1)$$

$$v_f = v_1 + sf(t_n + h, u_1), \quad u_f = u_1 + sf(t_n + h, v_f). \quad (2)$$

The physical output is the midpoint of the copies,

$$z_{n+1} = \frac{u_f + v_f}{2}.$$

“Midpoint” therefore refers to this arithmetic projection; the method is not the implicit-midpoint Runge–Kutta rule.

2 State and energy configurations

One endpoint-time A-B-B-A map followed by a spatial arithmetic mean. Energy tracking is optional and passive. The four accepted shear stages update the normalized passive momentum; no nonlinear solve is used. The common state conventions are specified below.

3 Symmetry, order, and geometry

The four shears form an explicit self-adjoint map on the duplicated phase space when the signed step reverses together with its endpoint times. Starting from the diagonal and taking the arithmetic mean yields a consistent symmetric method with designed global order two for sufficiently smooth dynamics.

The duplicated ABBA map is symplectic for its split Hamiltonian. Arithmetic projection, however, is not Hairer’s symmetric projection and does not in general make the closed physical map symplectic. The copy-separation quantity

$$d_n = \|u_f - v_f\|_\infty$$

is therefore retained as a diagnostic. It is neither an error estimator nor a convergence residual.

4 Work and time dependence

One step performs four vector-field evaluations. The evaluations at t_n and $t_n + h$ are part of the non-autonomous definition. The common fixed-grid runner chooses a uniform main grid independently of requested output times; off-grid output uses shadow steps that do not alter the accepted main trajectory or its diagnostics.

5 Implementation correspondence

The public class is implemented in `src/simulation/methods/abba/order2_midpoint.py`. It reuses the common endpoint-time stages from `abba/_core.py`; optional physical energy tracking lives in `abba/_energy.py`. The companion architecture document under `../simulation/` records the complete runtime and observation contract.

6 Four state formulations and passive energy tracking

For one planar particle the supported internal representations are

physical	$z = (x, y)$	$\mathbb{R}^2$,
physical with energy	(z, t, κ)	$\mathbb{R}^4$,
duplicated	$(u, v), \quad u, v \in \mathbb{R}^2$	$\mathbb{R}^4$,
duplicated with energy	(u, v, t, κ)	$\mathbb{R}^6$.

Classical methods use the physical rows; ABBA and BM4 use the duplicated rows. `track_energy` selects the energy variant. For N planar particles the respective dimensions are $2N$, $4N$, $4N$, and $6N$: one time entry and one momentum per particle. All time entries equal the integration time. Classical full-cyclotron runs instead use $4N$ or $6N$ entries. Accepted spatial copies coincide; only one physical copy is exported. Every coordinate block contains N values in component-major order. The formulation validates particle times and aligns output round-off; the integrator does not interpret coordinate indices.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, z), \quad \kappa(t_0) = 0, \quad \Delta K(t) = H(t, z(t)) + \kappa(t) - H(t_0, z_0).$$

Splitting accumulators from both copies are divided by two. The physical Hamiltonian may vary with time; ΔK measures its computed balance. Tracking leaves physical trajectories, nonlinear stopping rules and adaptive acceptance unchanged. Dynamics must supply the momentum derivative, including zero for an autonomous Hamiltonian.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired; `state_extension='fully_extended'` raises migration guidance.

Dynamics, formulation, method and integration remain separate. The shared state classes own layouts, diagonal embedding and history extraction in

`src/simulation/formulations/state.py`

Each integration initializes a fresh method instance. Observers receive physical states; energy histories are in `Solution.diagnostics`. The `extended_time` history has shape (N, n_{samples}) . Fixed-step shadow samples leave main states and counters unchanged. Adaptive energy quadrature uses accepted dense output; its backend state and Jacobian remain physical.